

Оглавление

Введение.....	4
Глава 1. Теоретические основы детекции LLM-текстов.....	7
1.1 Генеративные языковые модели и проблема происхождения текста.....	7
1.2 Подходы к детекции использования генеративных языковых моделей.....	9
1.2.1 Методы на основе перплексии и вероятностных характеристик.....	10
1.2.2 Методы на основе статистических и стилистических признаков текста..	11
1.2.3 Методы на основе применения методов машинного обучения.....	12
1.2.4 Обоснование выбора стилометрических метрик.....	13
1.2.5 Артефакты генерации.....	15
1.3 Обзор существующих систем детекции сгенерированных текстов.....	16
1.4 Выбор подхода для прототипа.....	18
Выводы к главе.....	18
Глава 2. Проектирование и реализация веб-сервиса аудита текста.....	20
2.1 Требования и архитектура системы.....	20
2.1.1 Функциональные требования.....	20
2.1.2 Компоненты и взаимодействие.....	21
2.1.3 Хранение данных и работа с базой данных.....	22
2.1.4 Контейнеризация и воспроизводимость запуска.....	24
2.1.5 Описание REST API.....	24
2.2 Реализация пайплайна обработки текста.....	25
2.2.1 Извлечение текста.....	25
2.2.2 Нормализация и вычисление метрик.....	26
2.3 Реализация скоринга и интеграция CatBoost.....	26
2.3.1 Схема использования модели в сервисе.....	27
2.3.2 Формирование и сохранение результатов аудита.....	27
2.4 Реализация веб-интерфейса.....	28
2.4.1 Структура страниц интерфейса.....	28
2.4.2 Сценарий работы пользователя на главной странице.....	29
2.4.3 Визуализация метрик и выгрузка отчета.....	29
2.4.4 Интерфейс аутентификации, профиль и экспертная оценка.....	30
2.5 Проверка работоспособности сервиса.....	30
Выводы к главе.....	31
Глава 3. Экспериментальное исследование и оценка качества детекции.....	32
3.1 Формирование исследовательского корпуса.....	32

3.1.1 Критерии отбора текстов и методология формирования данных.....	32
3.1.2 Корпус human-текстов.....	34
3.1.3 Корпус LLM-текстов.....	34
3.2 Формирование признакового пространства.....	35
3.3 Методика проведения эксперимента.....	36
3.3.1 Постановка задачи.....	36
3.3.2 Схема разбиения данных.....	36
3.3.3 Конфигурация CatBoost-классификатора.....	37
3.4 Метрики оценки качества.....	38
3.5 Результаты на тестовом фолде.....	39
3.6 Результаты кросс-валидации.....	40
3.7 Анализ важности признаков.....	41
3.8 Абляционный эксперимент.....	43
3.9 Сравнение с другими моделями.....	43
Выводы к главе.....	45
Заключение.....	46
Список использованных источников.....	48

Введение

Актуальность исследования обусловлена стремительным развитием генеративных языковых моделей (LLM), которое привело к существенному повышению качества автоматически создаваемых текстов. В результате тексты, сгенерированные с использованием таких моделей, становятся трудно отличимыми от текстов, написанных человеком, что создает риски в профессиональной среде, включая возможность появления недостоверных документов, нарушение авторских прав и распространение плагиата.

Существующие методы детекции машинно-сгенерированных текстов обладают рядом ограничений, в том числе высокой долей ложноположительных срабатываний при анализе формализованных профессиональных текстов и чувствительностью к параметрам генерации и последующего редактирования. Как показано в теоретическом обзоре, задача детекции носит вероятностный характер и требует применения комбинированных подходов.

Целью исследования является разработка и теоретическое обоснование прототипа сервиса аудита использования генеративных языковых моделей при создании профессиональных текстов.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести обзор и систематизацию существующих методов детекции текстов, сгенерированных с использованием языковых моделей.
2. Выделить ключевые признаки и метрики, применимые для анализа профессиональных жанров.
3. Спроектировать архитектуру сервиса аудита: определить компоненты, обмен данными и интерфейсы.
4. Реализовать набор алгоритмов для вычисления и агрегирования индикаторов.
5. Провести экспериментальную оценку прототипа на сформированном корпусе профессиональных текстов и измерить качество детекции.

6. Разработать рекомендации по интерпретации вероятностных оценок и сформировать предложения по масштабированию прототипа до уровня промышленного решения.

Под **аудитом** в рамках данной работы понимается комплексная процедура анализа текста, включающая вычисление признаков, оценку вероятности использования LLM и предоставление интерпретируемых диагностических данных, а не только бинарную классификацию.

Объектом исследования являются процесс автоматизированного анализа и выявления машинно-сгенерированных текстов в профессиональной среде.

Предметом исследования являются статистические и стилометрические признаки текстов, а также методы их агрегации и классификации для оценки вероятности использования генеративных языковых моделей при создании профессиональных текстов.

Научная новизна работы заключается в разработке и экспериментальной оценке гибридного подхода к аудиту профессиональных текстов, объединяющего статистические, стилометрические и Unicode-индикаторы с моделью машинного обучения для получения интерпретируемой оценки использования генеративных языковых моделей. В отличие от ряда существующих решений, ориентированных преимущественно на общие текстовые жанры, предложенный подход адаптирован к анализу профессиональных текстов.

Практическая значимость работы заключается в разработке функционирующего прототипа веб-сервиса, реализующего автоматизированный анализ документов, вычисление набора признаков и формирование вероятностной оценки использования LLM. Разработанная архитектура и предложенные принципы организации данных создают технологический фундамент для дальнейшей интеграции прототипа в контур цифровых сервисов образовательных организаций и перевода системы на уровень отказоустойчивого промышленного решения.

Методологическую основу исследования составляют методы статистического анализа текстов, методы машинного обучения и принципы инженерного проектирования программных систем.

Результатом работы является прототип веб-сервиса аудита текста, реализующий извлечение признаков, анализ Unicode-индикаторов и вычисление вероятностной оценки использования LLM, а также методика его экспериментальной проверки на корпусах human- и LLM-текстов.

Структура бакалаврской квалификационной работы.

Выпускная квалификационная работа состоит из введения, трех глав, заключения, списка литературы и приложения. Она содержит 57 страниц, 27 изображений и 9 таблиц.

Глава 1. Теоретические основы детекции LLM-текстов

1.1 Генеративные языковые модели и проблема происхождения текста

Генеративные языковые модели (LLM) представляют собой, как правило, авторегрессионные модели на основе архитектуры трансформеров, обученные предсказывать очередной токен в последовательности на основе предшествующего контекста. Например, GPT-модели являются авторегрессионными трансформерами [1]. Ключевой особенностью трансформерной архитектуры является механизм самовнимания (self-attention), позволяющий учитывать взаимосвязи между элементами входной последовательности и обеспечивающий параллельную обработку данных, в отличие от последовательной обработки в RNN и LSTM.

Поскольку современные LLM способны генерировать тексты, близкие по качеству к человеческим, возникает проблема определения происхождения текста.

Под определением происхождения текста понимается задача установления того, был ли текст написан человеком или сгенерирован с использованием языковой модели. Современные LLM обучаются на масштабных корпусах текстов и способны воспроизводить характерные свойства естественного языка, вследствие чего генерируемые тексты часто оказываются внешне неотличимыми от текстов, написанных человеком.

Основная трудность детекции обусловлена вероятностной природой генерации текста. Языковая модель формирует текст, последовательно выбирая токены в соответствии с распределением вероятностей, сформированным в процессе обучения. В результате генерируемые тексты обладают грамматической корректностью, логической связностью и статистическими характеристиками, близкими к человеческому стилю письма. С увеличением размера модели и объема обучающего корпуса точность воспроизведения языковых закономерностей возрастает.

Существенной проблемой для задач детекции является возможность последующего редактирования. Если текст, первоначально сгенерированный моделью, был частично отредактирован человеком, многие статистические признаки машинной генерации могут измениться настолько, что детекция будет невозможна.

Дополнительной сложностью детекции является шаблонность профессиональных текстов. Для них характерны ограниченный словарь и использование стандартных формулировок, что обуславливает высокую степень предсказуемости структуры. Вследствие этого такие тексты по своим статистическим характеристикам могут быть близки к текстам, сгенерированным генеративными языковыми моделями.

Перечисленные особенности существенно осложняют работу существующих методов детекции, которые также имеют ряд собственных ограничений.

Во-первых, методы, основанные на анализе вероятностных характеристик текста (например, перплексии или распределения вероятностей токенов), чувствительны к параметрам генерации и постобработки текста.

Во-вторых, методы машинного обучения требуют наличия размеченных данных, содержащих тексты как человека, так и языковых моделей. Такие классификаторы хорошо работают только в рамках тех жанров и моделей, на которых они были обучены. При появлении новых моделей или при попытке детекции текстов другого жанра, точность определения использования LLM будет снижена.

В-третьих, серьезной проблемой остаются ложные срабатывания. Это повышает вероятность ложной классификации формальных текстов как сгенерированных. Например, как отмечено в статье [2], изучение TOEFL-эссе показало около 60% ложноположительных срабатываний на текстах, написанных не-носителями языка, поскольку их упрощенный стиль совпадает с образцами LLM.

Таким образом, определение происхождения текста остается сложной задачей, для которой не существует универсального решения. На практике наиболее надежные системы используют комбинацию нескольких методов и выдают вероятностную оценку, а не категоричный вывод.

1.2 Подходы к детекции использования генеративных языковых моделей

В настоящее время предложено достаточно методов, направленных на выявление сгенерированных текстов. Эти методы основаны на различных свойствах текста – вероятностных характеристиках, статистических признаках лексики и синтаксиса, а также на применении методов машинного обучения и встроенных механизмов маркировки.

Существующие подходы можно условно разделить на несколько основных групп.

Первая группа методов основана на анализе вероятностных характеристик текста с использованием языковых моделей. В таких подходах оценивается, насколько вероятен данный текст с точки зрения модели. При этом используются такие показатели, как перплексия и распределение вероятностей слов.

Вторая группа методов использует статистические и стилометрические признаки текста. Эти признаки описывают структуру текста, разнообразие лексики, частотные характеристики слов и другие особенности, которые могут отличаться между человеческим и машинным текстом.

Третья группа связана с применением методов машинного обучения. В этом случае модель обучается различать тексты человека и тексты языковых моделей на основе размеченных данных. Для этой задачи используются как классические алгоритмы классификации, так и современные трансформерные модели, дообученные на специализированных корпусах.

Отдельное направление составляют методы цифровой маркировки («watermarking») текстов, сгенерированных с использованием LLM. В подобных подходах в процессе генерации текста встраиваются скрытые статистические

сигнатуры, позволяющие впоследствии определить факт использования языковой модели без анализа стилометрических особенностей текста. Несмотря на активные исследования таких подходов, их промышленное внедрение остается ограниченным. Например, по данным The Verge, один из лидеров ИИ-рынка, OpenAI, в августе 2024 года отказалась пометать водяными знаками тексты, сгенерированные с помощью ChatGPT [3].

Далее рассмотрим подробнее основные группы методов детекции использования генеративных языковых моделей.

1.2.1 Методы на основе перплексии и вероятностных характеристик

Перплексия отражает степень «неожиданности» последовательности для данной модели. Если текст легко предсказывается (низкая перплексия), то это признак машинной генерации [2]. Преимущество этого подхода в том, что результаты вычислений формально интерпретируемы и имеют строгую математическую основу. Однако эффективность детекторов на основе перплексии сильно зависит от выбора модели, токенизации и жанра текста. В работе К. Фрейзер описано, что для универсального детектора нужно калибровать порог на данных целевого жанра, поскольку универсального порога не существует [2]. Кроме того, получение перплексии для длинного текста может быть вычислительно дорогим, что ограничивает применение этого метода в реальном времени.

Так, для оценки времени вычисления перплексии по корпусу 100 текстов, использовалась языковая модель mGPT [4], размещенная на платформе Hugging Face [5] (ai-forever/mGPT, порядка $1,3 \cdot 10^9$ обучаемых параметров). Измерения выполнялись на аппаратной конфигурации с 8 ГиБ памяти типа unified memory; ускорение вычислений на GPU осуществлялось через подсистему Metal Performance Shaders (MPS) в составе PyTorch. По результатам измерений построены графики зависимости времени расчета перплексии от размера текста, представленные на рисунке 1.1.

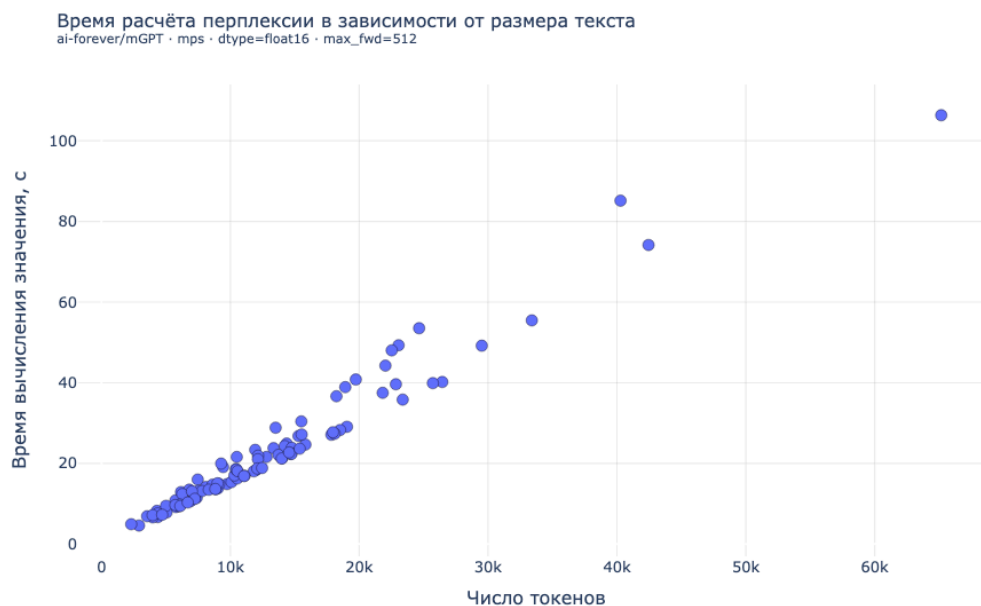


Рисунок 1.1 – зависимость времени вычисления перплексии от числа токенов

1.2.2 Методы на основе статистических и стилистических признаков текста

Вторая группа методов анализирует поверхностные и структурные характеристики текста:

- лексическое разнообразие, отражающее степень вариативности используемого словаря и позволяющее оценить богатство лексики текста;
- распределение длины слов и предложений, характеризующее структурные особенности текста и сложность синтаксических конструкций;
- энтропия текста, показывающая уровень неопределенности и разнообразия последовательности токенов;
- доля служебных (стоп-) слов, отражающая соотношение функциональной и содержательной лексики;
- пунктуационные характеристики, описывающие использование знаков препинания и структуру предложений;
- показатели повторяемости лексических единиц и фраз, позволяющие выявлять склонность к шаблонности и повторению.

Недостатки данного подхода связаны с ограниченной переносимостью между жанрами и доменами. Профессиональные тексты часто стандартизированы, из-за чего реальные и машинные тексты могут демонстрировать похожие стилистические профили. Это повышает риск ложных срабатываний при использовании только фиксированных эвристик.

Тем не менее, данный метод характеризуется существенно меньшей вычислительной сложностью. Так, для того же корпуса из 100 текстов, время вычислений кратно меньше по сравнению с расчетом перплексии. Результаты представлены на рисунке 1.2.

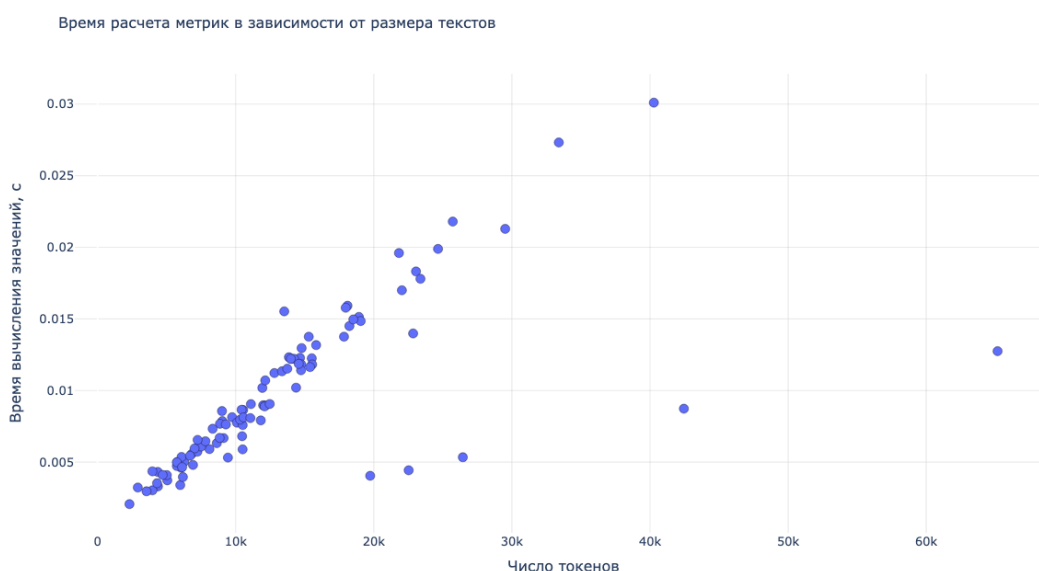


Рисунок 1.2 – время расчета метрик на корпусе из 100 текстов

1.2.3 Методы на основе применения методов машинного обучения

Методы данной группы предполагают использование обучаемых моделей для решения задачи классификации текстов по признаку их происхождения. В рамках данного подхода формируется размеченный корпус текстов, содержащий примеры как человеческих, так и машинно-сгенерированных документов, после чего выполняется обучение классификатора.

В качестве входных данных могут использоваться как заранее вычисленные признаки текста, так и непосредственно последовательности токенов текста.

К числу применяемых алгоритмов относятся классические методы машинного обучения, включая логистическую регрессию, метод опорных векторов и ансамблевые методы. В частности, градиентный бустинг на решающих деревьях (например, CatBoost), позволяет учитывать сложные нелинейные зависимости между признаками.

Кроме того, для задачи детекции активно применяются трансформерные архитектуры, такие как BERT, RoBERTa и DeBERTa, дообучаемые на специализированных корпусах [6]. Подобные модели способны автоматически выявлять сложные контекстные и стилистические закономерности, однако требуют значительных вычислительных ресурсов, больших объемов обучающих данных и характеризуются меньшей интерпретируемостью по сравнению с классическими моделями.

1.2.4 Обоснование выбора стилометрических метрик

Выбор метрик для разработки системы детектирования текстов обусловлен специфическими различиями в механизмах порождения речи человеком и большими языковыми моделями.

Лексическое разнообразие

Одной из ключевых метрик является лексическое разнообразие, так как языковые модели при генерации склонны к «регрессии к среднему». В то время как человек-специалист использует узкопрофильную терминологию и редкие синонимы, нейросети чаще выбирают наиболее вероятные (высокочастотные) токены из обучающей выборки. Как указывают Д. Ипполито и соавторы в работе «Automatic Detection of Generated Text» [7], это приводит к статистически значимому снижению количества редких слов в генерируемых текстах по сравнению с естественными.

Синтаксическая сложность

Анализ распределения вероятностей слов и структуры предложений позволяет выявить «стерильность» синтаксиса LLM. Человеческий стиль характеризуется «пульсацией», то есть естественным чередованием коротких и длинных, перегруженных конструкций. В исследовании С. Германа и соавторов [8] на примере системы GLTR показано, что модели чаще генерируют предложения средней длины, избегая синтаксических крайностей, что фиксируется инструментами статистической визуализации и анализа структуры.

Энтропия текста

Энтропия отражает уровень неопределенности и информационного разнообразия последовательности токенов. Для текстов, созданных LLM, характерна более низкая энтропия по сравнению с человеческой речью, так как алгоритмы генерации оптимизированы для выбора наиболее статистически вероятных продолжений фраз. С опорой на работу С. Германа [8] можно утверждать, что анализ предиктивной энтропии позволяет выявить «машинный след»: генеративные модели стремятся к минимизации информационных «сюрпризов», что делает их структуру избыточно предсказуемой.

Показатели повторяемости лексических единиц и фраз

Данные метрики позволяют выявить склонность к шаблонности и циклическому использованию паттернов. Согласно исследованию Д. Ипполито [7], из-за специфики декодирования (например, top-k или nucleus sampling) модели могут демонстрировать избыточную фиксацию на определенных лексических конструкциях. Мониторинг повторов n-грамм помогает зафиксировать эффект «заикливания» и низкой вариативности, которая нехарактерна для качественных человеческих текстов, стремящихся к избеганию тавтологии.

Использование служебных слов

Распределение функциональных слов (союзов, предлогов, местоимений) рассматривается как уникальный «цифровой отпечаток», который почти не

контролируется сознанием автора. Э. Стаматос в фундаментальном обзоре современных методов атрибуции авторства [9] подтверждает, что частотность функциональных слов является наиболее стабильным признаком авторства. Использование n-грамм и частотных профилей позволяет отличить профиль «усредненного интернета», характерный для LLM, от индивидуального стиля человека.

Пунктуационная плотность и технические признаки

Включение в анализ пунктуационных паттернов и спецсимволов (включая Unicode-сигналы) необходимо для обеспечения состоятельной устойчивости системы. Технические манипуляции, такие как замена символов или специфическое использование авторских знаков препинания, часто применяются для обхода автоматических детекторов. Э. Крозерс и соавторы в работе «Adversarial Robustness of Neural-Statistical Features» [10] доказывают, что мониторинг таких аномалий позволяет эффективно выявлять попытки сокрытия машинного происхождения текста.

1.2.5 Артефакты генерации

Помимо статистических и стилометрических признаков, важным индикатором генерации текста являются специфические технические артефакты. Эти цифровые следы возникают на этапе декодирования токенов в текст или в процессе работы системных инструкций модели по форматированию вывода.

К основным категориям таких артефактов относятся:

1. Неразрывные пробелы и *bidirectional*-маркеры;
2. Шаблоны экранирования.

Использование узкого неразрывного пробела (NNBSP U+202F) или специфических маркеров направления письма (*bidirectional*-маркеры) часто нехарактерно для ручного набора текста в стандартных текстовых редакторах, однако

встречается в выдаче моделей, обучавшихся на определенных корпусах веб-данных. Эти символы могут попадать в итоговый текст при копировании из веб-интерфейсов чат-ботов.

Характерной чертой последних поколений моделей является специфическое экранирование служебных символов, таких как знаки доллара `\$` или тильды `\~`. Это связано с внутренней логикой обработки Markdown и LaTeX-разметки в системных промптах моделей, которые стремятся предотвратить некорректный рендеринг формул или кода.

Однако Unicode-артефакты рассматриваются не как устойчивый признак генерации, а как дополнительный индикатор, способный повысить чувствительность системы в отдельных сценариях копирования текста из интерфейсов LLM-сервисов.

1.3 Обзор существующих систем детекции сгенерированных текстов

Развитие генеративных моделей привело к появлению ряда программных решений, нацеленных на идентификацию машинного контента. Современные детекторы можно разделить на две группы: специализированные онлайн-сервисы и модули, интегрированные в системы проверки заимствований.

GPTZero

Один из наиболее известных инструментов, разработанный Эдвардом Тианом (Принстонский университет). Согласно официальной методологии компании, описанной А. Барлоу [11], в основе работы алгоритмов лежат две метрики – перплексия и *burstiness*. Достоинством системы является наглядная визуализация «индекса человечности» для каждого предложения. Несмотря на то, что сервис развит и поддерживается уже долго время, ложные срабатывания возникают в довольно очевидных местах, таких как оглавление, например, как показано на рисунке 1.3 (Приложение 1).

Copyleaks

Коммерческое решение, использующее многослойные классификаторы. В отличие от открытых инструментов, Copyleaks не раскрывает конкретные метрики, однако в документации API [12] указано, что система анализирует «статистические паттерны» и сопоставляет их с историческими данными генерации. Система ориентирована на корпоративный и образовательный сегменты, предлагая интеграцию через API.

Модуль детекции ИИ-текстов в системе «Антиплагиат»

Ведущее российское решение для академической среды. Согласно официальному описанию функционала [13], модуль анализирует структуру документа и выявляет паттерны использования популярных LLM (ChatGPT, Gemini и др.). Система не предоставляет пользователю расчетные значения стилометрических признаков, выдавая лишь итоговый вердикт о наличии признаков генерации.

Сравнительный анализ систем

Для систематизации особенностей существующих решений в таблице 1.1 (Приложение 1) приведено их сравнение по ключевым для данной работы параметрам.

Проведенный обзор показывает, что популярные существующие системы являются закрытыми проприетарными решениями. Это затрудняет их использование для глубокого анализа того, на основании каких именно признаков был вынесен вердикт. Кроме того, существующие решения редко предоставляют доступ к расчетным стилометрическим характеристикам, что ограничивает возможность экспертной оценки в спорных случаях. Разрабатываемый прототип призван восполнить этот пробел, предоставляя пользователю полный набор параметров для качественной интерпретации оценки детекции.

1.4 Выбор подхода для прототипа

Для разработки прототипа в данной работе выбран гибридный подход, основанный на использовании статистических и стилометрических признаков текста в сочетании с методами машинного обучения. В рамках данного подхода из текста извлекается набор количественных характеристик, на основе которых классификатор формирует вероятностную оценку принадлежности текста к классу сгенерированных с использованием генеративных моделей.

Выбор данного подхода обусловлен рядом факторов. Во-первых, используемые признаки обеспечивают интерпретируемость результатов и позволяют анализировать вклад отдельных характеристик текста в итоговую оценку. Во-вторых, табличные модели машинного обучения эффективно учитывают нелинейные зависимости между признаками и при этом остаются относительно простыми в реализации и интеграции в веб-сервис.

Рабочая гипотеза исследования заключается в том, что комбинация статистических и стилометрических признаков текста с обучаемым классификатором позволяет получать устойчивую вероятностную оценку использования генеративных языковых моделей при создании профессиональных текстов.

Проверка гипотезы осуществляется в экспериментальной части работы на размеченных корпусах текстов с использованием стандартных метрик качества классификации, включая ROC-AUC, precision, recall, F1-score и accuracy.

Выводы к главе

Анализ теоретических основ показал, что высокая адаптивность современных трансформерных моделей делает задачу детекции текстов трудновыполнимой без использования комплексных подходов.

Существующие коммерческие системы, такие как GPTZero и «Антиплагиат», функционируют как закрытые решения, не позволяющие эксперту проанализировать конкретные причины вынесенного вердикта. В связи с этим в

работе обоснован выбор гибридного метода на основе машинного обучения и явных стилометрических признаков. Данный подход позволяет компенсировать вычислительную дороговизну вероятностных методов и обеспечить полную прозрачность расчетных метрик, что критически важно для качественной интерпретации результатов в профессиональной среде.

Глава 2. Проектирование и реализация веб-сервиса аудита текста

2.1 Требования и архитектура системы

2.1.1 Функциональные требования

Разработанная система предназначена для аудита текстовых документов и получения интерпретируемой вероятностной оценки использования генеративных языковых моделей при их написании.

Архитектура сервиса проектировалась с учетом следующих функциональных требований:

1. Извлечение текстового содержимого из документов распространенных форматов.
2. Прием текста через веб-интерфейс, запуск единого пайплайна предобработки и расчета признаков.
3. Вычисление статистических и стилометрических метрик, используемых как входные признаки модели машинного обучения и как интерпретируемые показатели для пользователя.
4. Формирование вероятностной оценки с использованием обученной модели машинного обучения.
5. Выявление Unicode-символов и следов экранирования, которые могут указывать на копирование текста из интерфейса чатов LLM-сервисов.
6. Сохранение истории проверок, формирование отчетов и предоставление преподавателю возможности зафиксировать экспертную оценку результата.
7. Поддержка регистрации, входа в систему и разграничение ролей пользователя: студент и преподаватель.

Таким образом, система должна обеспечивать полный прикладной цикл: от загрузки документа и расчета признаков до сохранения результата, просмотра отчета и сбора экспертной обратной связи.

2.1.2 Компоненты и взаимодействие

В качестве основного языка разработки выбран Python, поскольку он обладает развитой экосистемой для решения задач машинного обучения и построения веб-сервисов. Серверная часть реализована на FastAPI. Данный фреймворк позволяет описывать REST API, передавать сессии базы данных в обработчики запросов через механизм внедрения зависимостей и одновременно обслуживать HTML-страницы демонстрационного интерфейса. Веб-интерфейс реализован с использованием шаблонов Jinja2, что позволило сохранить простую монолитную архитектуру без выделения отдельного фронтенд-приложения.

Сервис построен по модульному принципу. Основные этапы обработки вынесены в отдельные слои:

1. ingestion – извлечение текста из документов разных форматов;
2. preprocessing – очистка и нормализация текста перед расчетом признаков;
3. metrics – вычисление статистических, стилометрических и технических характеристик;
4. scoring – загрузка модели машинного обучения и расчет вероятности класса LLM.
5. report_builder – формирование структурированного отчета для интерфейса и API;
6. db – сохранение пользователей, проверок, отчетов и экспертных оценок.

Типовой сценарий взаимодействия компонентов выглядит следующим образом. Пользователь вводит текст вручную или загружает документ на главной странице. Если загружен файл одного из поддерживаемых форматов, клиентская часть сначала вызывает маршрут POST /extract, после чего полученный текст может быть очищен и передан в аудит. Основной маршрут POST /audit запускает анализ текста, сохраняет проверку в базе данных, формирует полный отчет и возвращает JSON-ответ с идентификатором проверки и ссылкой на страницу отчета. Далее пользователь может открыть отчет по маршруту /report/{id},

просмотреть историю проверок или, если он имеет роль преподавателя, сохранить экспертную оценку результата.

Рисунок 2.1 (Приложение А) показывает границы системы и связь пользовательского интерфейса, API, базы данных и обученной модели.

2.1.3 Хранение данных и работа с базой данных

В текущей версии сервиса в качестве хранилища используется реляционная СУБД SQLite. Такой выбор обусловлен простотой развертывания: база данных хранится в одном файле, не требует отдельного серверного процесса и подходит для локального, учебного и демонстрационного запуска. По умолчанию файл базы данных создается в каталоге `data/app.db` корневой папки проекта. Строка подключения задается переменной окружения `DATABASE_URL`.

Доступ к данным реализован через `SQLModel` – библиотеку, объединяющую типизированные модели `Pydantic` и возможности `SQLAlchemy ORM`. Таблицы описаны в виде Python-классов, а работа с данными в маршрутах `FastAPI` выполняется через зависимость `get_session`. Для каждого HTTP-запроса создается отдельный контекст сессии, внутри которого выполняются операции чтения и записи, после чего сессия закрывается.

Инициализация схемы выполняется при старте приложения вызовом `SQLModel.metadata.create_all`. Для SQLite дополнительно включается поддержка ограничений внешних ключей с помощью `PRAGMA foreign_keys=ON`. Поскольку `create_all` создает отсутствующие таблицы, но не изменяет структуру уже существующих, в приложении предусмотрены точечные процедуры актуализации SQLite-схемы для ранее созданных баз данных.

Логическая схема базы данных представлена на рисунке 2.2. В ней выделены сущности:

1. *users* – учетные записи пользователей системы. В таблице хранятся электронная почта, имя и фамилия пользователя, хэш-значение пароля, роль в системе (*student* или *teacher*), флаг активности учетной записи и дата

регистрации. Эта сущность используется для JWT-аутентификации и разграничения доступа к API.

2. *audit_checks* – история выполненных проверок текстов. Каждая запись содержит UUID проверки, ссылку на пользователя, имя исходного файла, время проверки, значение P(LLM) и полный отчет в поле *report_json*. Отчет хранится как JSON-снимок результата, что позволяет восстановить страницу отчета и API-ответ без повторного пересчета текста.
3. *teacher_audit_feedback* – экспертная обратная связь от преподавателя. Запись фиксирует согласие или несогласие с результатом детекции (*agrees_with_detection*). Уникальное ограничение на пару *audit_check_id* и *teacher_id* гарантирует, что один преподаватель оставляет не более одной оценки по одной проверке. Дополнительно в таблице сохраняется снимок имени файла, P(LLM) и метрик на момент оценки.

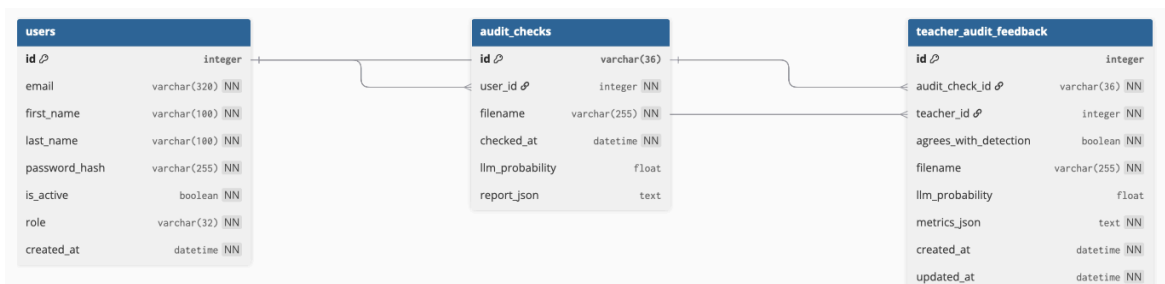


Рисунок 2.2 – Схема БД

Связи между пользовательскими данными настроены через внешние ключи с каскадным удалением. При удалении пользователя удаляются связанные проверки и экспертные оценки, что упрощает поддержание целостности данных.

Компактная схема достаточна для прототипа и демонстрационного развертывания. При необходимости перехода к серверной СУБД (например, PostgreSQL) базовый слой доступа к данным может быть сохранен: потребуется изменить *DATABASE_URL* и учесть особенности промышленной миграции схемы.

2.1.4 Контейнеризация и воспроизводимость запуска

Для обеспечения воспроизводимого запуска в проекте используется Docker. Файл Dockerfile описывает сборку образа на базе python:3.11-slim, установку зависимостей из requirements.txt, копирование кода приложения и запуск ASGI-сервера uvicorn на порту 8000. Такой подход фиксирует программное окружение и снижает риск ошибок, связанных с различием локальных версий Python и библиотек.

Для удобного запуска сервиса подготовлен docker-compose.yml. Он выполняет сборку образа, пробрасывает порт 8000:8000, задает переменные окружения и монтирует каталог models/catboost внутрь контейнера в режиме только для чтения. В compose-конфигурации по умолчанию включена переменная ENABLE_CATBOOST=1, а путь к модели задается через CATBOOST_MODEL_PATH. Переменная JWT_SECRET используется для подписи токенов аутентификации.

Использование Docker Compose позволяет запустить сервис одной командой и получить одинаковую структуру окружения как на локальной машине, так и при переносе приложения на сервер.

2.1.5 Описание REST API

REST API сервиса разделено на несколько групп маршрутов. Маршруты аутентификации (/api/auth/register, /api/auth/login, /api/auth/me) отвечают за регистрацию, вход, получение и изменение данных текущего пользователя. Маршрут POST /extract используется для извлечения текста из загруженного документа. Маршрут POST /thesis-clean выполняет эвристическую очистку текста ВКР перед анализом.

Основной вычислительный маршрут POST /audit принимает файл с текстом в формате UTF-8, требует JWT-аутентификации и возвращает рассчитанные

метрики, значение `llm_probability`, блок `hidden_unicode`, идентификатор сохраненной проверки и ссылку на отчет. Маршруты `/api/audit-history` и `/api/reports` предоставляют доступ к истории проверок и сохраненным отчетам пользователя. Для преподавателей реализованы маршруты `/api/teacher-feedback`, позволяющие сохранить, получить и просмотреть экспертные оценки по проверкам.

Полное описание эндпоинтов REST API приведено в приложении Б (таблица Б.1).

2.2 Реализация пайплайна обработки текста

2.2.1 Извлечение текста

Извлечение содержимого документов вынесено в отдельные модули для каждого поддерживаемого формата и доступно через маршрут `POST /extract`. Пользовательский интерфейс вызывает этот маршрут при загрузке файла на главной странице, после чего полученный текст подставляется в поле анализа. Такой подход отделяет задачу чтения документов от собственно аудита текста и позволяет использовать извлечение как вспомогательный этап.

Поддерживаются следующие форматы:

1. PDF – извлечение через `pympdf4llm`;
2. DOCX – чтение с использованием библиотеки `python-docx`;
3. RTF – обработка через `stripptf`;
4. ODT – извлечение средствами `odfpy`;
5. TXT – прямое декодирование содержимого в UTF-8.

Для форматов, требующих обработки внешними библиотеками, файл временно сохраняется на диск, передается соответствующему модулю извлечения, а затем удаляется. Это исключает накопление пользовательских документов в файловой системе. Результат работы маршрута имеет единый вид: JSON-объект с полем `text`, содержащим извлеченный текст.

2.2.2 Нормализация и вычисление метрик

Маршрут POST /audit инициирует основной процесс анализа текста. На первом этапе исходная строка проверяется на наличие скрытых Unicode-символов и следов экранирования. Это важно, поскольку часть диагностических признаков может исчезнуть после очистки текста. На данном этапе фиксируются скрытые и служебные символы Unicode, включая неразрывные пробелы, символы нулевой ширины и маркеры направления письма, а также последовательности экранирования служебных символов, например знак доллара и тильды.

На втором этапе выполняется базовая нормализация текста функцией `clean_text`: приведение к нижнему регистру, сведение последовательностей пробельных символов к одному пробелу и удаление пробелов по краям строки. После этого рассчитывается набор числовых метрик:

1. `lexical_diversity` – лексическое разнообразие;
2. `burstiness` – вариативность длин предложений;
3. `average_sentence_length` – средняя длина предложения;
4. `text_entropy` – энтропия текста;
5. `stop_word_ratio` – доля стоп-слов;
6. `word_length_variation` – вариативность длины слов;
7. `punctuation_ratio` – доля пунктуационных символов;
8. `repetition_score` – показатель повторяемости;
9. `unicode` – бинарный диагностический признак наличия Unicode-сигналов.

Пайплайн обработки текста изображен в виде схемы на рисунке 2.3 в Приложении В.

2.3 Реализация скоринга и интеграция CatBoost

После расчета метрик они передаются в модуль `scoring`, отвечающий за получение итоговой вероятности `llm_probability`. В качестве основной модели

используется CatBoost-классификатор, обученный на табличном признаковом описании текстов. Использование модели CatBoost в сервисе управляется переменной окружения `ENABLE_CATBOOST`: при значении 1 приложение загружает модель и рассчитывает вероятность $P(\text{LLM})$. Если переменная не включена, файл модели отсутствует или зависимость CatBoost недоступна, поле `llm_probability` принимает значение `null`, а остальные диагностические данные продолжают возвращаться пользователю.

При этом API остается работоспособным: метрики, Unicode-анализ, сохранение проверки и формирование отчета продолжают выполняться.

2.3.1 Схема использования модели в сервисе

Взаимодействие с моделью CatBoost реализовано в отдельном адаптере `catboost_runtime`. Адаптер определяет путь к файлу модели, загружает `catboost_model.cbm`, считывает список признаков из файла `inference.json` и формирует входной вектор в нужном порядке. По умолчанию модель находится по пути `models/catboost/catboost_model.cbm`; при необходимости путь может быть переопределен переменной окружения `CATBOOST_MODEL_PATH`.

Рисунок 2.4 (Приложение Г) демонстрирует схему интеграции модели в общий процесс обработки запроса.

2.3.2 Формирование и сохранение результатов аудита

Результат аудита возвращается в виде JSON-объекта. Основные поля ответа:

1. `metrics` – рассчитанные числовые признаки текста;
2. `llm_probability` – вероятность класса LLM или `null`, если CatBoost недоступен;
3. `hidden_unicode` – диагностический блок со сведениями о скрытых Unicode-символах и экранированиях;
4. `check_id` – идентификатор сохраненной проверки;

5. `check` – краткая информация о записи истории;
6. `report_id` и `report_url` – идентификатор и ссылка на полный отчет.

Дополнительно на сервере формируется расширенный отчет. Он содержит сведения о документе, краткое решение по порогу 0,5, строки метрик со сравнением с эталонными параметрами human-корпуса, Unicode-сводку и служебную информацию о включении CatBoost и перплексии. Отчет сохраняется в поле `report_json` таблицы `audit_checks` и доступен через `/api/reports/{report_id}` и страницу `/report/{report_id}`.

2.4 Реализация веб-интерфейса

Для демонстрации работы алгоритма реализован веб-интерфейс на серверных шаблонах Jinja2. Он предоставляет пользователю полный сценарий работы с документом: загрузку или ввод текста, предварительное извлечение содержимого, запуск аудита, просмотр краткой сводки, визуализацию метрик, открытие сохраненного отчета и переход к истории проверок.

2.4.1 Структура страниц интерфейса

Пользовательский интерфейс включает следующие страницы:

1. `/` – главная страница аудита текста;
2. `/history` – история проверок авторизованного пользователя;
3. `/report/{report_id}` – страница сохраненного отчета;
4. `/login` и `/register` – страницы входа и регистрации;
5. `/account` – личный кабинет и редактирование данных пользователя;
6. `/info` – информационная страница с пояснениями по сервису.

Базовая навигация и общие стили задаются в шаблоне `base.html`, что обеспечивает единообразие интерфейса и повторное использование общих элементов.

2.4.2 Сценарий работы пользователя на главной странице

На главной странице пользователь может вставить текст вручную или загрузить документ. При загрузке документа интерфейс вызывает POST /extract, получает извлеченный текст и подставляет его в поле ввода. Для ВКР дополнительно предусмотрена очистка текста через POST /thesis-clean: она удаляет часть служебных фрагментов документа и оставляет основной текст для дальнейшего анализа.

После нажатия кнопки «Посчитать метрики» интерфейс отправляет текст на POST /audit. Пользователь получает краткую сводку: значение P(LLM), если модель доступна, способ расчета, наличие или отсутствие Unicode-сигналов и ссылку на сохраненный отчет (Приложение Д, рисунок 2.6). Ниже отображается полный JSON-ответ API, что делает результат прозрачным и пригодным для отладки или документирования (Приложение Д, рисунок 2.7).

Таким образом, интерфейс объединяет операционный и диагностический уровни: пользователь видит как итоговую оценку, так и исходные числовые признаки.

2.4.3 Визуализация метрик и выгрузка отчета

Для повышения интерпретируемости данных реализована визуализация метрик в виде SVG-графиков, формируемых на клиентской стороне (Приложение Д, рисунок 2.8).

Графики отображают сравнение значений анализируемого текста с эталонными параметрами корпуса, что позволяет оценить отклонения по отдельным признакам.

После выполнения проверки пользователь может открыть полный отчет по ссылке. Отчет хранится в базе данных и остается доступен через историю

проверок (Приложение Д, Рисунок 2.9), поэтому результат можно повторно просматривать без нового запуска анализа. Кроме HTML-страницы предусмотрен JSON-доступ через `/api/reports/{report_id}`, что позволяет использовать результаты сервиса во внешних процессах или сохранять их для последующего анализа. Пример отчета, который был загружен с использованием такого метода, можно увидеть на рисунке 2.10 в Приложении Д.

2.4.4 Интерфейс аутентификации, профиль и экспертная оценка

Пользовательский интерфейс включает страницы регистрации (Приложение Д, рисунок 2.11), входа (Приложение Д, рисунок 2.12) и личного кабинета (Приложение Д, рисунок 2.13). При регистрации указывается роль пользователя: студент или преподаватель. После входа JWT-токен используется клиентской частью при обращении к защищенным маршрутам API. В личном кабинете (рисунок 2.13) пользователь может просматривать и изменять данные учетной записи, а также удалить аккаунт.

История проверок доступна авторизованному пользователю на странице `/history`. Из нее можно перейти к сохраненным отчетам. Для преподавателей в интерфейс отчета и главной страницы добавлен блок экспертной оценки: преподаватель может отметить, согласен ли он с результатом детекции. Эта оценка сохраняется в `teacher_audit_feedback` вместе со снимком `P(LLM)` и метрик на момент принятия решения, что позволяет в дальнейшем использовать такие данные для анализа качества модели и возможного расширения обучающей выборки.

2.5 Проверка работоспособности сервиса

Для проверки корректности реализации в проекте используются автоматизированные тесты на базе `pytest`. Тесты охватывают основные серверные сценарии: регистрацию и вход пользователя, изменение и удаление учетной

записи, запуск аудита, сохранение истории проверок, получение отчетов, разграничение доступа по ролям, а также сохранение экспертной оценки преподавателя. Результат выполнения представлен на рисунке 2.14 в Приложении Д: все 63 теста завершились успешно.

При тестировании используется изолированная SQLite-база в памяти (`sqlite:///memory:`), что позволяет запускать проверки без изменения рабочей базы данных приложения. Такой подход обеспечивает воспроизводимость тестов и позволяет контролировать корректность пользовательских сценариев после изменений в коде.

Выводы к главе

Во второй главе были сформулированы требования к прототипу и описана реализация веб-сервиса аудита текста. Архитектура построена на FastAPI, SQLAlchemy, SQLite и серверных шаблонах Jinja2, что обеспечивает простоту развертывания и достаточную модульность для дальнейшего развития проекта.

Реализован полный пайплайн обработки: извлечение текста из документов, базовая нормализация, вычисление статистических и стилометрических признаков, диагностический анализ Unicode-сигналов, интеграция CatBoost и формирование сохраненных отчетов. Отдельное внимание уделено хранению истории проверок и экспертной обратной связи преподавателя, поскольку эти данные могут использоваться для последующей оценки качества детекции.

Дополнительно работоспособность ключевых серверных сценариев проверена автоматизированными тестами, запускаемыми через pytest.

Таким образом, разработанный прототип представляет собой не только демонстрационный классификатор, но и прикладной веб-сервис, поддерживающий сценарии загрузки документов, интерпретации результата, повторного просмотра отчетов и сбора экспертных оценок.

Глава 3. Экспериментальное исследование и оценка качества детекции

3.1 Формирование исследовательского корпуса

Экспериментальная часть основана на анализе двух типов текстовых данных: оригинальных выпускных квалификационных работ (далее – human), опубликованные в открытом доступе, и синтетических текстов, сгенерированных большой языковой моделью на основе метаданных тех же работ (далее – LLM). В данном разделе описаны критерии отбора данных, пайплайн построения корпуса и процедура формирования табличного набора признаков для обучения и оценки классификатора.

Итоговый датасет содержит 997 объектов: 414 human-текстов (label = 0) и 583 LLM-текста (label = 1). Такое распределение классов является умеренно несбалансированным: доля LLM-текстов выше доли human-текстов, однако дисбаланс не является критическим. Для корректной оценки качества использовались стратифицированные разбиения, сохраняющие соотношение классов в обучающих и тестовых частях.

В данном разделе описаны критерии формирования корпусов, процедура генерации LLM-текстов, построение признакового пространства и подготовка данных для обучения классификатора.

3.1.1 Критерии отбора текстов и методология формирования данных

Цель экспериментальной части – проверить, обеспечивает ли выбранный набор признаков в сочетании с табличным классификатором устойчивое разделение текстов классов human и LLM в профессиональном жанре.

Для достижения сопоставимости выборок использованы критерии отбора, указанные в таблице 3.1.

Таблица 3.1 – Критерии формирования корпуса

Критерий	Содержание и обоснование
Жанровая сопоставимость	Использование текстов академического и научно-технического стиля.
Тематическая близость	Отбор работ из области компьютерных наук и информационных технологий.
Сопоставимость объёма	Для LLM-корпуса целевой размер генерации задаётся с ориентиром на объём исходного human-текста.
Воспроизводимость	Фиксация параметров генерации (API, температура) и единый пайплайн препроцессинга.
Исключение утечки между выборками	При разбиении используется группировка по <code>work_id</code> , чтобы тексты одной работы не попадали одновременно в обучение и тест.

Особое внимание уделялось предотвращению утечки данных. Так как human-текст и связанный с ним LLM-текст могут относиться к одной исходной работе, простое случайное разбиение объектов могло бы привести к завышенной оценке качества. Поэтому при обучении и проверке использовался идентификатор `work_id`: один и тот же `work_id` не должен одновременно присутствовать в обучающей и тестовой выборке.

На рисунке 3.1 (Приложение Е) изображена схема ML-пайплайна, которая описывает последовательность преобразований данных от момента их получения до выгрузки обученной модели.

3.1.2 Корпус human-текстов

В качестве основы класса human (label = 0) использованы оригинальные тексты ВКР из открытых источников. Для каждого текста в датасете фиксировался идентификатор работы work_id, целевая метка и рассчитанные числовые признаки. Сами тексты не передавались непосредственно в классификатор: модель обучалась только на агрегированных статистических и стилометрических характеристиках. Полный текст используется только на этапе извлечения признаков.

3.1.3 Корпус LLM-текстов

Класс LLM (label = 1) формировался путем генерации текстов по метаданным оригинального корпуса. На основе названия и аннотации каждой работы создавался индивидуальный промпт, показанный на рисунке 3.2. Целевой объем генерации рассчитывался как функция от длины исходного человеческого текста, что обеспечило сопоставимость масштабов выборки.

```
def build_prompt(*, title: str, abstract: str, body_text: str) -> str:
    wc = _word_count(body_text or "")
    target = 2500 if wc <= 0 else max(1500, min(wc, 4500))
    lo, hi = int(target * 0.9), int(target * 1.1)
    return (
        "Сгенерируй новый оригинальный текст ВКР на русском языке.\n"
        "Требования:\n"
        "- Академический стиль, техническая направленность.\n"
        "- НЕ копируй дословно существующие тексты.\n"
        "- Структура: (1) раздел «Введение» с постановкой цели и обзором дальнейшего; "
        "(2) главы 1, 2 и 3 с содержательным текстом; после каждой из трёх глав – "
        "краткий подраздел с выводами по этой главе (несколько предложений или короткий абзац, без дублирования всей главы).\n"
        "- Без: реферата, abstract, общего заключения всей работы в конце, списка источников, приложений.\n"
        "- Связный текст, без лишних маркдаун-разметок.\n"
        f"- Объем: около {target} слов всего (допустимо {lo}-{hi}); доведи до диапазона, "
        "без обрыва на середине раздела.\n\n"
        f"Тема: {title.strip()}\n\n"
        f"Аннотация (из исходной работы):\n{abstract}\n\n"
        "Верни только текст работы."
    )
```

Рисунок 3.2 – Целевой промпт

Для генерации текстов использовалось API DeepSeek со следующими параметрами:

1. Модель deepseek-chat;

2. Температура 1.0;
3. Лимит вывода – 8192 токенов;

При обрыве генерации по лимиту длины выполнялось до 10 последовательных итераций продолжения контекста.

Результаты сохранялись в структурированную таблицу `llm_works`, включающую текст генерации, параметры модели и метаданные запроса.

Связь объектов через `work_id` сохранялась и при построении обучающей таблицы. Благодаря этому при валидации можно исключить ситуацию, когда модель фактически проверяется на тексте, тематически связанном с уже увиденной работой. Групповая схема разбиения делает оценку качества более строгой и ближе к прикладному сценарию, в котором сервис получает новую, ранее не встречавшуюся работу.

3.2 Формирование признакового пространства

Для обоих корпусов применялся единый алгоритм расчета метрик, согласованный с модулем анализа веб-сервиса. На этапе экспериментов рассчитывался расширенный набор признаков: статистические и стилометрические показатели, Unicode-индикатор и, в отдельных конфигурациях, перплексия. Такой расширенный набор использовался для первичного анализа информативности признаков.

Финальная модель, интегрированная в веб-сервис, использует компактное признаковое пространство из восьми статистических и стилометрических признаков:

1. `lexical_diversity`;
2. `burstiness`;
3. `average_sentence_length`;
4. `text_entropy`;
5. `stop_word_ratio`;
6. `word_length_variation`;

7. punctuation_ratio;
8. repetition_score.

Из входного вектора модели исключены work_id, label и unicode. Поле work_id является служебным и необходимо только для разбиения данных. Поле label является целевой переменной. Поле perplexity в текущем наборе данных полностью состоит из пропусков и поэтому не используется. Unicode-анализ сохранён как диагностический блок, но не включён во входной вектор CatBoost-классификатора.

Таким образом, финальная модель является табличным классификатором по восьми компактным признакам.

3.3 Методика проведения эксперимента

3.3.1 Постановка задачи

Эксперимент рассматривается как задача бинарной классификации. На вход классификатора подаётся вектор из восьми числовых признаков, рассчитанных по тексту. На выходе модель оценивает вероятность принадлежности объекта к классу LLM (label = 1). При стандартном пороге 0,5 объект относится к одному из двух классов:

- label = 0 – текст, написанный человеком;
- label = 1 – текст, сгенерированный LLM.

Для оценки качества использовались как метрики ранжирования, так и метрики классификации при фиксированном пороге. Это важно, поскольку ROC-AUC показывает способность модели разделять классы по вероятностной оценке, а accuracy, precision, recall и F1 отражают качество конкретных бинарных решений.

3.3.2 Схема разбиения данных

Для контроля качества применялось стратифицированное групповое разбиение `StratifiedGroupKFold` с числом фолдов `n_splits = 5`, перемешиванием и фиксированным `random_state = 42`. Стратификация сохраняет соотношение классов, а группировка по `work_id` предотвращает попадание связанных объектов в разные части одного разбиения.

Для детального анализа результатов использовался один out-of-fold тестовый фолд (таблица 3.2)

Таблица 3.2

Часть выборки	Размер таблицы	Размер матрицы признаков
Train	797 x 12	797 x 8
Test	200 x 12	200 x 8

Пересечение `work_id` между `train` и `test` равно 0, что подтверждает корректность группового разбиения.

Дополнительно была проведена 5-fold кросс-валидация. В каждом фолде внешний test-фолд использовался только для оценки качества, а внутри обучающей части выделялось 15% групп `work_id` под валидацию для ранней остановки. Такая схема снижает риск переобучения и позволяет оценить устойчивость модели на разных разбиениях.

3.3.3 Конфигурация CatBoost-классификатора

В качестве основной модели выбран `CatBoostClassifier`. Выбор CatBoost обусловлен тем, что градиентный бустинг по деревьям хорошо подходит для табличных признаков, устойчив к признакам разного масштаба и позволяет получать интерпретируемые оценки важности признаков.

Конфигурация модели указана в таблице 3.3:

Таблица 3.3

Параметр	Значение
Алгоритм	CatBoostClassifier
iterations	800
learning_rate	0.05
depth	6
loss_function	Logloss
eval_metric	AUC
random_seed	42
early_stopping_rounds	50

Финальная модель обучалась с выделением 15% групп work_id под внутреннюю валидацию для ранней остановки. Лучшая итерация составила 177.

3.4 Метрики оценки качества

Для оценки качества использовались следующие метрики:

1. ROC-AUC – интегральная мера разделяющей способности модели, не зависящая от конкретного порога классификации;
2. ассигасу – доля правильно классифицированных объектов;
3. precision – доля верных срабатываний среди объектов, отнесённых моделью к выбранному классу;
4. recall – доля найденных объектов выбранного класса среди всех объектов этого класса;

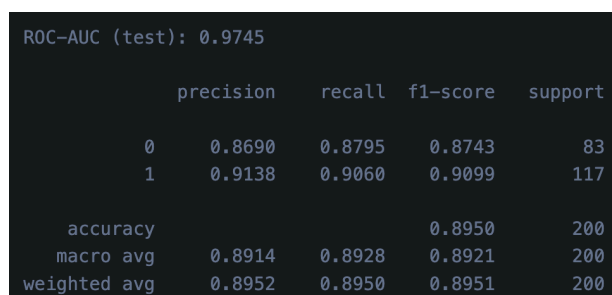
5. F1-score – гармоническое среднее precision и recall;
6. матрица ошибок – абсолютное число верных и ошибочных решений по каждому классу.

В прикладном сценарии интерпретация ошибок особенно важна.

Ложноположительный результат для human-текста означает, что человеческий текст ошибочно отнесён к LLM. Ложноотрицательный результат для LLM-текста означает, что сгенерированный текст не был обнаружен. Поэтому наряду с агрегированными метриками необходимо анализировать распределение ошибок между классами.

3.5 Результаты на тестовом фолде

На out-of-fold тестовом фолде объёмом 200 объектов модель показала высокое качество классификации. Значение ROC-AUC составило 0,9745, что указывает на хорошую способность модели ранжировать human- и LLM-тексты по вероятности принадлежности к классу LLM. Результаты проиллюстрированы на рисунке 3.3.



	precision	recall	f1-score	support
0	0.8690	0.8795	0.8743	83
1	0.9138	0.9060	0.9099	117
accuracy			0.8950	200
macro avg	0.8914	0.8928	0.8921	200
weighted avg	0.8952	0.8950	0.8951	200

Рисунок 3.3 – Результаты на тестовом фолде

Общая точность составила 0,8950, то есть 179 из 200 объектов были классифицированы верно. Для класса LLM модель показала precision 0,9138 и recall 0,9060. Это означает, что модель достаточно уверенно выделяет сгенерированные тексты и при этом не сводит задачу к механическому выбору большинства.

Для класса human recall составил 0,8795. Следовательно, часть человеческих текстов ошибочно относится к LLM, однако масштаб этой ошибки ограничен: из 83 human-текстов тестового фолда 73 были распознаны верно.

Матрица ошибок представлена на рисунке 3.4.

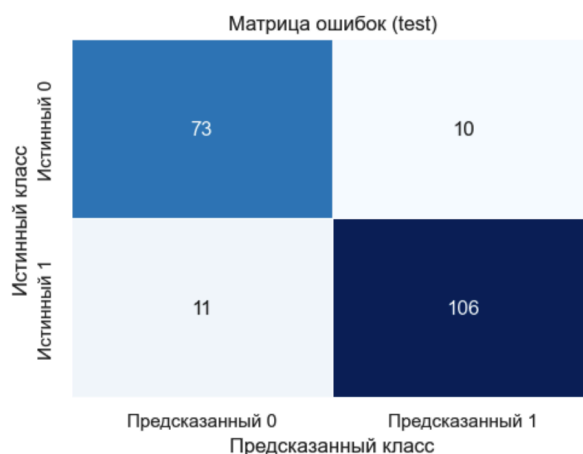


Рисунок 3.4 – Матрица ошибок на тестовом фолде

Из матрицы ошибок видно, что модель допустила 21 ошибку: 10 human-текстов были ошибочно отнесены к LLM, а 11 LLM-текстов были ошибочно отнесены к human. Ошибки распределены почти симметрично, что является положительным результатом: модель не демонстрирует выраженного перекоса в сторону одного из классов.

С практической точки зрения такой результат можно считать устойчивым для исследуемого корпуса. При этом вывод модели должен рассматриваться как аналитическая подсказка, а не как безусловное доказательство происхождения текста. Особенно это важно для ложноположительных срабатываний на human-текстах.

3.6 Результаты кросс-валидации

Для проверки устойчивости результата была проведена 5-fold кросс-валидация со стратификацией по целевой переменной и группировкой по

work_id. В каждом фолде модель обучалась на обучающей части, внутри которой дополнительно выделялся валидационный набор групп для ранней остановки. Тестовый фолд не пересекался с обучающей частью по work_id. Результаты представлены на рисунке 3.5.

	fold	roc_auc	accuracy	f1_macro	best_iteration	n_test
0	0	0.968283	0.890000	0.887468	140	200
1	1	0.981670	0.945000	0.943262	143	200
2	2	0.973853	0.925000	0.922035	31	200
3	3	0.980618	0.924242	0.921501	93	198
4	4	0.974969	0.924623	0.921771	172	199

Рисунок 3.5 – Результаты кросс-валидации

Средние значения и стандартные отклонения приведены в таблице 3.4.

Таблица 3.4

Метрика	Среднее значение	Стандартное отклонение
ROC-AUC	0,9759	0,0054
Accuracy	0,9218	0,0198
F1 macro	0,9192	0,0200

Средний ROC-AUC 0,9759 при стандартном отклонении 0,0054 показывает, что разделяющая способность модели сохраняется на разных фолдах. Ассигасу варьируется сильнее, от 0,8900 до 0,9450, что ожидаемо для тестовых фолдов объёмом около 200 объектов. Тем не менее среднее значение ассигасу 0,9218 и F1 macro 0,9192 подтверждают, что модель устойчиво работает для обоих классов.

3.7 Анализ важности признаков

Для интерпретации модели использовались два способа оценки важности признаков:

1. встроенная важность признаков CatBoost;
2. permutation importance по падению ROC-AUC.

Встроенная важность показывает, какие признаки активно использовались моделью при построении ансамбля деревьев. Permutation importance даёт более прикладную проверку: значение признака случайно перемешивается, после чего измеряется падение качества. Если после перемешивания конкретного признака ROC-AUC заметно снижается, значит этот признак действительно важен для классификации.

Встроенная важность CatBoost

Встроенная важность признаков CatBoost представлена в таблице 3.5 (Приложение Ж).

Наибольшую встроенную важность имеет `lexical_diversity`. Это означает, что модель часто использует лексическое разнообразие при разделении human- и LLM-текстов. Вторым по значимости является `word_length_variation`, отражающий вариативность длины слов. Остальные признаки также имеют ненулевой вклад, поэтому решение модели формируется не по одному показателю, а по совокупности стилометрических характеристик.

Permutation importance

Для дополнительной проверки была рассчитана permutation importance. В отличие от встроенной важности CatBoost, этот метод напрямую показывает, насколько ухудшается качество модели при нарушении информации в отдельном признаке. В качестве метрики качества использовался ROC-AUC.

Результаты permutation importance (Приложение Ж, таблица 3.6) подтверждают ведущую роль `lexical_diversity`: при перемешивании этого признака ROC-AUC снижается в среднем примерно на 0,196. Это наиболее сильное падение среди всех признаков. Вторым по влиянию является `word_length_variation`, при

перемешивании которого ROC-AUC снижается примерно на 0,049. Остальные признаки оказывают меньший, но также измеримый вклад.

3.8 Абляционный эксперимент

Так как оба способа анализа важности показали доминирующую роль `lexical_diversity`, была выполнена дополнительная проверка: модель обучалась повторно без этого признака. Цель эксперимента – убедиться, что качество классификации не определяется только одним показателем.

Результаты сравнение модели со всеми признаками и модели без `lexical_diversity` представлены в таблице 3.7, Приложение Ж.

Удаление `lexical_diversity` привело к снижению ROC-AUC на 0,0078. Снижение качества есть, но оно не является критическим: модель без ведущего признака всё равно сохраняет высокую разделяющую способность. Следовательно, остальные признаки также содержат полезную информацию о различиях между human- и LLM-текстами, а качество классификации не является результатом одного изолированного признака.

3.9 Сравнение с другими моделями

Для обоснования выбора CatBoost выполнено сравнение с рядом альтернативных моделей на том же датасете и с тем же протоколом валидации: `StratifiedGroupKFold` по полю `work_id` (5 фолдов, `random_state=42`), чтобы связанные тексты одной работы не попадали одновременно в обучение и проверку. В сравнении участвовали логистическая регрессия, случайный лес, Extra Trees, HistGradient Boosting и CatBoost с гиперпараметрами, согласованными с основным экспериментом. Результаты кросс-валидации по ROC-AUC представлены в таблице 3.8, Приложение Ж.

На hold-out первого фолда (200 объектов) лидируют HistGradient Boosting и Extra Trees. Разница с лучшими моделями по среднему ROC-AUC в CV составляет

около 0,004; парный критерий Уилкоксона для CatBoost и Extra Trees (5 фолдов) даёт $p = 0,0625$, то есть статистически значимого преимущества одной из моделей при уровне 0,05 не выявлено.

Для оценки устойчивости моделей проведен кросс-валидационный эксперимент (7 значений `random_state` $\times$ 5 фолдов). Результаты сравнения среднего ROC-AUC для ансамблевых методов представлены в таблице 3.8 (Приложение Д). Анализ показывает, что CatBoost демонстрирует сопоставимую с аналогами точность (ROC-AUC 0,9742) и входит в группу наиболее эффективных моделей, при этом не являясь абсолютным лидером по данной метрике

Отдельно измерена скорость инференса. Медианная латентность одного запроса: CatBoost – около 0,13 мс, логистическая регрессия – около 0,32 мс, HistGradient Boosting – около 6 мс, случайный лес и Extra Trees – около 11 мс. При параллельной обработке (8 потоков) CatBoost обеспечивает порядка 6–6,3 тыс. запросов/с, что на порядок выше, чем у лесовых моделей.

Результаты сравнений представлены на рисунках 3.6 – 3.10 в Приложении Ж.

По итогам сравнения CatBoost не превосходит другие ансамбли, однако демонстрирует сопоставимое качество при существенно более высокой скорости работы, компактном бинарном формате .cbm, фиксированном списке признаков и едином пайплайне.

Выводы к главе

В ходе экспериментального исследования была построена и проверена модель детекции LLM-текстов в академическом домене. Итоговый датасет содержит 997 объектов: 414 human-текстов и 583 LLM-текста. Для обучения финальной модели использовались восемь статистических и стилометрических признаков, рассчитанных единым пайплайном.

Основной моделью выбран CatBoostClassifier с функцией потерь Logloss и контролем качества по ROC-AUC. Для предотвращения утечки данных использовалась группировка по work_id: связанные объекты не попадали одновременно в train и test.

На тестовом фолде объёмом 200 объектов модель достигла ROC-AUC 0,9745 и accuracy 0,8950. Матрица ошибок показала 179 верных классификаций из 200: 73 верно распознанных human-текста, 106 верно распознанных LLM-текстов, 10 ложноположительных и 11 ложноотрицательных решений. Ошибки распределены почти симметрично, что свидетельствует об отсутствии выраженного перекоса модели в сторону одного класса.

Кросс-валидация подтвердила устойчивость результата: средний ROC-AUC составил 0,9759 при стандартном отклонении 0,0054, средняя accuracy - 0,9218, средний F1 macro - 0,9192. Анализ важности признаков показал ведущую роль lexical_diversity и word_length_variation, а абляционный эксперимент без lexical_diversity подтвердил, что остальные признаки также несут значимый сигнал.

Полученные результаты позволяют сделать вывод, что компактный набор статистических и стилометрических признаков в сочетании с CatBoost-классификатором обеспечивает высокое качество разделения human- и LLM-текстов в исследуемом академическом корпусе.

Заключение

В рамках выпускной квалификационной работы был разработан прототип сервиса аудита использования генеративных языковых моделей при создании профессиональных текстов. Актуальность исследования подтверждается стремительным ростом качества генерации LLM, что затрудняет идентификацию происхождения текстов в академической и профессиональной среде.

Для достижения поставленной цели в ходе работы были успешно решены следующие задачи:

1. **Обоснован выбор гибридного подхода** к детекции, сочетающего расчет интерпретируемых стилометрических признаков (лексических, синтаксических и пунктуационных) с применением алгоритмов машинного обучения.
2. **Спроектирован и разработан веб-сервис**, обеспечивающий полный цикл аудита: от автоматизированного извлечения текста из различных форматов документов до визуализации результатов. Реализованный пайплайн включает расчет статистических метрик, анализ аномальных Unicode-индикаторов и модуль скоринга на базе алгоритма CatBoost.
3. **Проведена экспериментальная оценка**, подтвердившая высокую эффективность предложенного подхода: классификатор продемонстрировал значение ROC-AUC 0,9745 при accuracy 0,8950, показав при этом практически полное отсутствие ложноположительных срабатываний на текстах, написанных человеком.
4. **Разработаны практические рекомендации** по интерпретации результатов работы системы.

По итогам исследования были зафиксированы объективные ограничения разработанного подхода: зависимость точности от жанровой специфики данных и чувствительность к последующему редактированию текста человеком. В связи с этим итоговая оценка сервиса носит строго вероятностный характер. Результат

работы системы следует рассматривать исключительно как инструмент аналитической поддержки, помогающий эксперту принять обоснованное решение, а не как безапелляционное доказательство машинного происхождения контента.

Перспективы дальнейшего развития проекта и его масштабирования до уровня промышленного решения связаны с глубокой модернизацией сбора данных и непрерывного обучения модели. Вектор развития предполагает интеграцию разработанного веб-сервиса с внутренними информационными системами университета. Автоматизация пайплайна загрузки и обработки документов из официальных академических хранилищ позволит непрерывно расширять обучающую выборку текстами, в том числе обучая модель для работы с другими предметными областями.

В технологическом плане для перехода к промышленной эксплуатации планируется миграция с СУБД SQLite на кластер PostgreSQL для надежного ведения истории отчетов и накопления экспертной обратной связи. Качество детекции планируется улучшать за счет автоматического переобучения модели на новых данных.

Таким образом, цель выпускной квалификационной работы полностью достигнута: разработан, теоретически обоснован и экспериментально проверен прототип веб-сервиса, пригодный для практического использования в задачах контроля качества и аудита профессиональных текстов, а также сформирован фундамент для его дальнейшего развития в отказоустойчивую промышленную систему.

Список использованных источников

1. Lee, Minhyeok A Mathematical Interpretation of Autoregressive Generative Pre-Trained Transformer and Self-Supervised Learning // Mathematics. 2023. Vol. 11, iss. 11. Art. 2451 / Minhyeok Lee. — Текст : электронный // MDPI : [сайт]. — URL: <https://www.mdpi.com/2311698> (дата обращения: 15.05.2026).
2. Fraser, Kathleen C. F. Detecting AI-Generated Text: Factors Influencing Detectability with Current Methods / K. C. Fraser, H. Dawkins, S. Kiritchenko. — Текст : электронный // arXiv : [сайт]. — URL: <https://arxiv.org/html/2406.15583v2> (дата обращения: 15.05.2026).
3. Розанова, А. OpenAI отказалась помечать тексты, сгенерированные ChatGPT. Этот шаг отвернул бы от компании почти 30% пользователей / А. Розанова. — Текст : электронный // РБК Life : [сайт]. — URL: <https://rbc.ru/life/news/66b092299a79471714ebbad2> (дата обращения: 16.05.2026).
4. Shliazhko, O. mGPT: Few-Shot Learners Go Multilingual / O. Shliazhko, A. Fenogenova, M. Tikhonova. — Текст : электронный // arXiv : [сайт]. — URL: <https://arxiv.org/abs/2204.07580> (дата обращения: 16.05.2026).
5. ai-forever/mGPT : Multilingual GPT model. — Текст : электронный // Hugging Face : [сайт]. — URL: <https://huggingface.co/ai-forever/mGPT> (дата обращения: 16.05.2026).
6. Wang, H. AI-Generated Text Detection and Classification Based on BERT Deep Learning Algorithm / H. Wang, J. Li, Z. Li. — Текст : электронный // arXiv : [сайт]. — URL: <https://arxiv.org/abs/2405.16422> (дата обращения: 16.05.2026).
7. Ippolito, D. Automatic Detection of Generated Text is Easiest when Humans are Fooled / D. Ippolito, D. Duckworth, C. Callison-Burch, D. Eck. — Текст : электронный // Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. — Б. м. : Association for Computational Linguistics, 2020. — С. 1808-1822. — URL: <https://aclanthology.org/2020.acl-main.164/> (дата обращения: 16.05.2026).

8. Gehrmann, S. GLTR: Statistical Detection and Visualization of Generated Text / S. Gehrmann, H. Strobelt, A. M. Rush. — Текст : электронный // Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. — Florence, Italy : Association for Computational Linguistics, 2019. — С. 111-116. — URL: <https://aclanthology.org/P19-3019/> (дата обращения: 16.05.2026).
9. Stamatatos, E. A survey of modern authorship attribution methods / E. Stamatatos. — Текст : электронный // Journal of the American Society for Information Science and Technology. — 2009. — № 3. — С. 538-556. — URL: <https://icsdweb.aegean.gr/stamatatos/papers/survey.pdf> (дата обращения: 16.05.2026).
10. Crothers, E. Adversarial Robustness of Neural-Statistical Features in Detection of Generative Transformers / E. Crothers. — Текст : электронный // ResearchGate : [сайт]. — URL: https://www.researchgate.net/publication/359254281_Adversarial_Robustness_of_Neural-Statistical_Features_in_Detection_of_Generative_Transformers (дата обращения: 17.05.2026).
11. Barlow, A. <https://gptzero.me/news/how-ai-detectors-work/> / A. Barlow. — Текст : электронный // GPTZero : [сайт]. — URL: <https://gptzero.me/news/how-ai-detectors-work/> (дата обращения: 17.05.2026).
12. AI Text Detection API. — Текст : электронный // CopyleaksDocs : [сайт]. — URL: <https://docs.copyleaks.com/concepts/products/ai-text-detection-api/> (дата обращения: 17.05.2026).
13. Китарь, Н. Главные функции системы «Антиплагиат» / Н. Китарь. — Текст : электронный // Антиплагиат : [сайт]. — URL: <https://antiplagiat.ru/blog/main-functions-of-anti-plagiarism/> (дата обращения: 17.05.2026).
14. Bao, G. Fast-DetectGPT: Efficient Zero-Shot Detection of Machine-Generated Text via Conditional Probability Curvature / G. Bao. — Текст : электронный //

arXiv : [сайт]. — URL: <https://arxiv.org/abs/2310.05130> (дата обращения: 17.05.2026).

15. How does Copyleaks AI Detection work?. — Текст : электронный // Copyleaks : [сайт]. — URL: <https://help.copyleaks.com/s/article/HowdoesCopyleaksAIDetectionwork681cc74da8fd1> (дата обращения: 17.05.2026).